

StudyFlowAI

AI Tools & Usage Documentation

This document separates the two roles AI played: AI that helped **build** the product, and AI that **runs inside** it. One rule held throughout: AI was the accelerator, never the author of record — every suggestion was reviewed, run and tested by me before it entered the repository.

1 · AI used to BUILD the product

Tool: Arena.ai Agent Mode (coding agent), used as a pair programmer directed by me — architecture options, scaffolding, feature drafts, debugging hypotheses, test and documentation drafts. Google AI Studio docs and MDN for primary-source verification; no AI design tools (the UI is hand-rolled Tailwind). **Ground rules:** no secrets in prompts; real errors pasted back verbatim; nothing committed without running it myself.

How the work was divided:

- **AI drafted** modules, tests and docs — **I reviewed, edited and ran** everything
- **I decided** architecture and trade-offs — AI provided options and pushback
- **Debugging:** AI hypothesized; I reproduced and verified every root cause
- **Deploys, quota budgeting, verification, the demo — 100% me**

Why this discipline: several bugs found during the build were in AI-drafted code that looked correct on review — which is why the observability and test layers were built first, not last.

2 · AI used BY the product

All provider access goes through one gateway (`backend/src/ai/gemini.js`): every call is recorded to an `aicalls` collection (feature, model, latency, tokens, success) and every output is validated against a JSON schema before use. Model IDs are environment config.

Feature	Model	Structured output
Document concepts	gemini-3.6-flash	[[name, summary, sourcePage]]
Embeddings (index + retrieval)	gemini-embedding-001	768-d vectors, RETRIEVAL_DOCUMENT / _QUERY
Tutor answers	gemini-3.6-flash	{answer, insufficientEvidence, usedSources[]}
Quiz generation	gemini-3.6-flash	[[type, concept, difficulty, prompt, options, answer, keyPoints, sourcePage]]
Open-answer grading	gemini-3.6-flash	{score, feedback, understood[], missed[]}

Deliberately NOT AI: concept selection (database query sorted by mastery), mastery updates (fixed 50/50 blend, unit-tested), recommendations (rule on the weakest concept, with a stated reason), and the “no evidence → refuse” gate (similarity threshold checked before any model call). Deterministic, explainable, free — putting an LLM where a sorted query belongs is not “more AI”, it is less engineering.

3 · Rules every AI call follows

1. **Tracked** — one record per call (feature, model, latency, tokens, success, error); logging failures never breaks the call
2. **Retried** — transient 429/5xx retried with exponential backoff; permanent failures surface immediately
3. **Validated** — schema-constrained output, then normalized: types coerced, defaults applied, malformed items dropped with a logged reason
4. **Untrusted** — document content and user messages are data, never instructions (prompt-injection defence)
5. **Values over indexes** — the model returns the correct option's TEXT; code derives the index (models cannot count reliably)

4 · How AI behaviour was evaluated

Automated, offline, every change: 15 tests / 6 suites against an in-memory database with the AI replaced by controllable stand-ins — auth, cross-user isolation, chunking, ranking, the full pipeline on a real 2-page PDF, and the quiz loop including the exact mastery math (30 → 15).

- grounded questions cite real material + page; reworded questions still retrieve — verified in messages.citations
- out-of-scope questions refuse BEFORE the model — aicalls shows no generation call, zero cost
- rubric-hit open answers score ≥ 80 with 'understood' naming the points; gaps named in 'missed'
- adversarial: malformed quiz items dropped and logged (5 kept / 1 dropped); provider outage → retries, then a clean 503

Cost is watched, not just correctness: aicalls aggregates (calls, failures, latency, tokens per feature) are reviewed in the admin dashboard after every prompt or retrieval change. Free-tier design: batched embeddings (one call per PDF), refusal short-circuit, one call per quiz — a full demo run costs 6–8 generations.

Honest gap: no automated LLM-as-judge evaluations in CI yet — the curated checks above are its manual stand-in — automating them in CI is the natural next step.

5 · Known AI limitations

- free tier: 20 generations/day/model, occasional overload — handled, not eliminated
- retrieval is chunk-level similarity ($\leq \sim 1k$ chunks), no re-ranking
- refusal threshold (similarity ≥ 0.3) is a constant, not per-project calibrated
- quiz difficulty is prompt-guided, not a psychometric model